

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 03

Студент: Стелина Петрити

Группа: НПИбд-02-21

Код:

```
#include <SFML/Graphics.hpp>
#include <iostream>
#include <cmath>
#include <fstream>

// Константа PI для удобства
const double PI = 3.14159265358979323846;

// Класс, представляющий трехмерные координаты точки наблюдения
class viewpiont {
public:
    double x, y, z;
    viewpiont() : x(0), y(0), z(0) {}
    viewpiont(double X, double Y, double Z) : x(X), y(Y), z(Z) {}
};

// Класс, представляющий вершину в трехмерном пространстве
class threedvertex {
public:
    viewpiont global; // Координаты вершины в мировой системе координат
    viewpiont view; // Координаты вершины в системе координат наблюдения

    threedvertex() {}

    // Метод для установки координат вершины
    void clvertex(viewpiont g) {
        global = g;
    }

    // Метод для преобразования координат вершины в систему наблюдения
    void coordinates(viewpiont coord) {
        // Распаковка углов поворота из coord
        double roangle = coord.x, tetangle = coord.y, fiangle = coord.z;
        // Вычисление новых координат вершины в системе координат наблюдения
        view.x = -global.x * sin(tetangle) + global.y * cos(tetangle);
        view.y = -global.x * cos(fiangle) * cos(tetangle) - global.y * cos(fiangle) *
sin(tetangle) + global.z * sin(fiangle);
        view.z = -global.x * sin(fiangle) * cos(tetangle) - global.y * sin(fiangle) *
sin(tetangle) - global.z * cos(fiangle) + roangle;
    }
};

// Класс, представляющий ребро между двумя вершинами
class edge {
private:
    threedvertex tmpvertex, fervertex;

public:
    edge() {}

    // Метод для установки вершин, соединенных ребром
    void edgech(threedvertex svertex, threedvertex fvertex) {
        tmpvertex = svertex;
        fervertex = fvertex;
    }
}
```

```

// Метод для отрисовки ребра
void edgedrawing(double l, sf::RenderWindow& window) {
    l = l / 2;
    // Вычисляем координаты начальной точки линии в окне отображения
    double x1 = (int)((l * tmpvertex.view.x) / tmpvertex.view.z);
    double y1 = (int)((l * tmpvertex.view.y) / tmpvertex.view.z);
    // Вычисляем координаты конечной точки линии в окне отображения
    double x2 = (int)((l * ferververtex.view.x) / ferververtex.view.z);
    double y2 = (int)((l * ferververtex.view.y) / ferververtex.view.z);
    // Создаем массив вершин для линии
    sf::Vertex line[] = {
        sf::Vertex(sf::Vector2f(x1 + (window.getSize().x / 2), y1 +
(window.getSize().y / 2))),
        sf::Vertex(sf::Vector2f(x2 + (window.getSize().x / 2), y2 +
(window.getSize().y / 2))),
    };
    // Рисуем линию на окне
    window.draw(line, 2, sf::Lines);
}
};

// Структура, представляющая ребро по индексам вершин
struct structure {
public:
    int X, Y;
};

// Класс, представляющий методы для рисования
class drawingmethod {
private:
    threedvertex* vertives; // Массив vertice
    edge* edges; // Массив edges
    structure* structure1; // Массив пар индексов вершин
    viewpiont viewpoint; // Точка наблюдения
    bool c = 0; // Флаг, указывающий на выполнение определенных операций

public:
    int a, b;

    // Метод для отрисовки графа
    void drawgraph(sf::RenderWindow& window) {
        for (int i = 0; i < b; i++)
            // Для каждого ребра вызываем метод отрисовки edgedrawing
            edges[i].edgedrawing(viewpoint.x, window);
    }

    // Метод для считывания данных из файла
    void drawingfigure(const std::string& filename) {
        int X, Y, Z;
        if (!c) {
            std::ifstream file(filename);
            if (file.is_open()) {
                file >> a;
                vertives = new threedvertex[a];
                for (int i = 0; i < a; i++) {
                    file >> X >> Y >> Z;
                    viewpiont p = viewpiont(X * 100, Y * 100, Z * 100);
                }
            }
        }
    }
};

```

```

        vertives[i].clvertex(p);
        vertives[i].coordinates(viewpoint);
    }
    file >> b;
    edges = new edge[b];
    structure1 = new structure[b];
    for (int i = 0; i < b; i++) {
        file >> X >> Y;
        structure1[i].X = X;
        structure1[i].Y = Y;
        edges[i].edgech(vertives[structure1[i].X],
vertives[structure1[i].Y]);
    }
    c = true;
}
}
for (int i = 0; i < a; i++)
    vertives[i].coordinates(viewpoint);
for (int i = 0; i < b; i++)
    edges[i].edgech(vertives[structure1[i].X], vertives[structure1[i].Y]);
}

// Метод для установки точки наблюдения
void setviewpoint(double rotation, double thetaangle, double phiangle) {
    viewpoint.x = rotation;
    viewpoint.y = thetaangle;
    viewpoint.z = phiangle;
}
};
int main() {
    double rotation = 200, thetaangle = 90, phiangle = 120;

    // Создание окна SFML
    sf::RenderWindow window(sf::VideoMode(800, 600), "SFML graphic figures");
    drawingmethod cube;

    // Установка точки наблюдения и считывание данных из файла
    cube.setviewpoint(rotation, thetaangle, phiangle);
    cube.drawingfigure("cube.dat");//("tetrahedron.dat")

    // Основной цикл приложения
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        window.clear();

        // Изменение точки наблюдения и повторная отрисовка графа
        cube.setviewpoint(rotation, thetaangle += (PI / 18), phiangle += (PI / 18));
        cube.drawingfigure("cube.dat");//("tetrahedron.dat")
        cube.drawgraph(window);

        window.display();
        sf::sleep(sf::milliseconds(250));
    }
}

```

```
    return 0;  
}
```

cube.dat

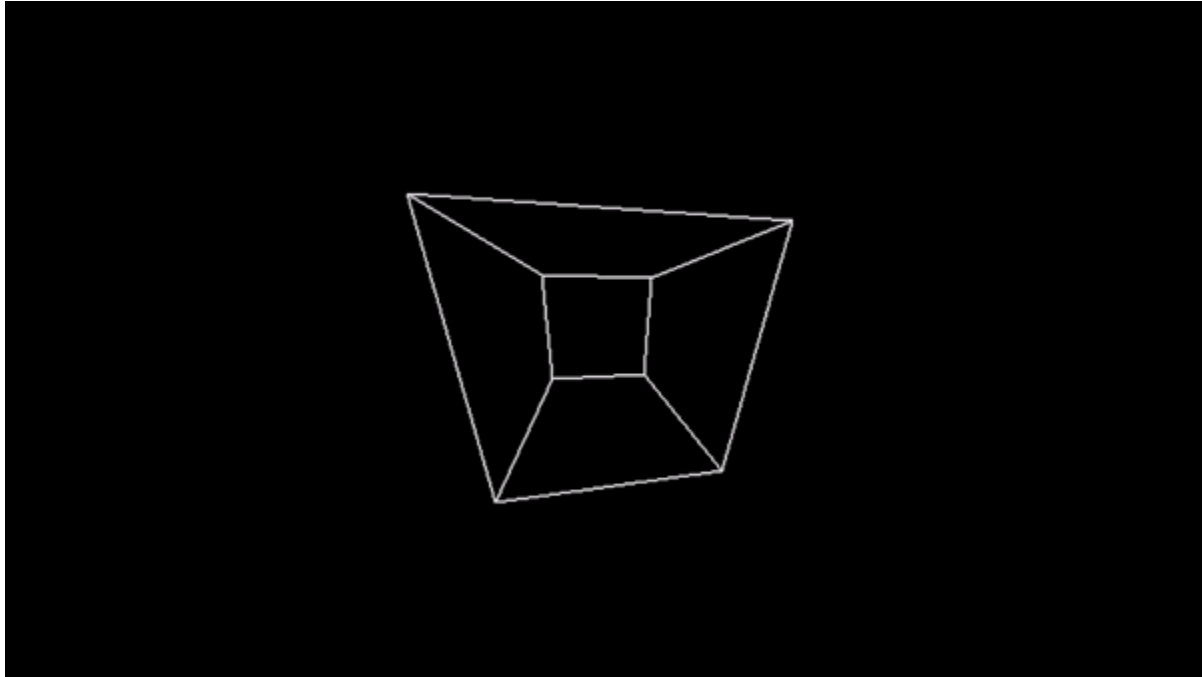
```
8  
-1 -1 -1  
-1 1 -1  
1 1 -1  
1 -1 -1  
-1 -1 1  
-1 1 1  
1 1 1  
1 -1 1  
12  
0 1  
0 3  
0 4  
2 1  
2 3  
2 6  
5 1  
5 4  
5 6  
7 3  
7 4  
7 6
```

tetrahedron.dat

```
4  
-1 -1 -1  
1 -1 -1  
0 1 -1  
0 0 1  
6  
0 1  
1 2  
2 0  
0 3  
1 3  
2 3
```

Вывод:

GIF КУБ



GIF ТЕТРАЭДР

